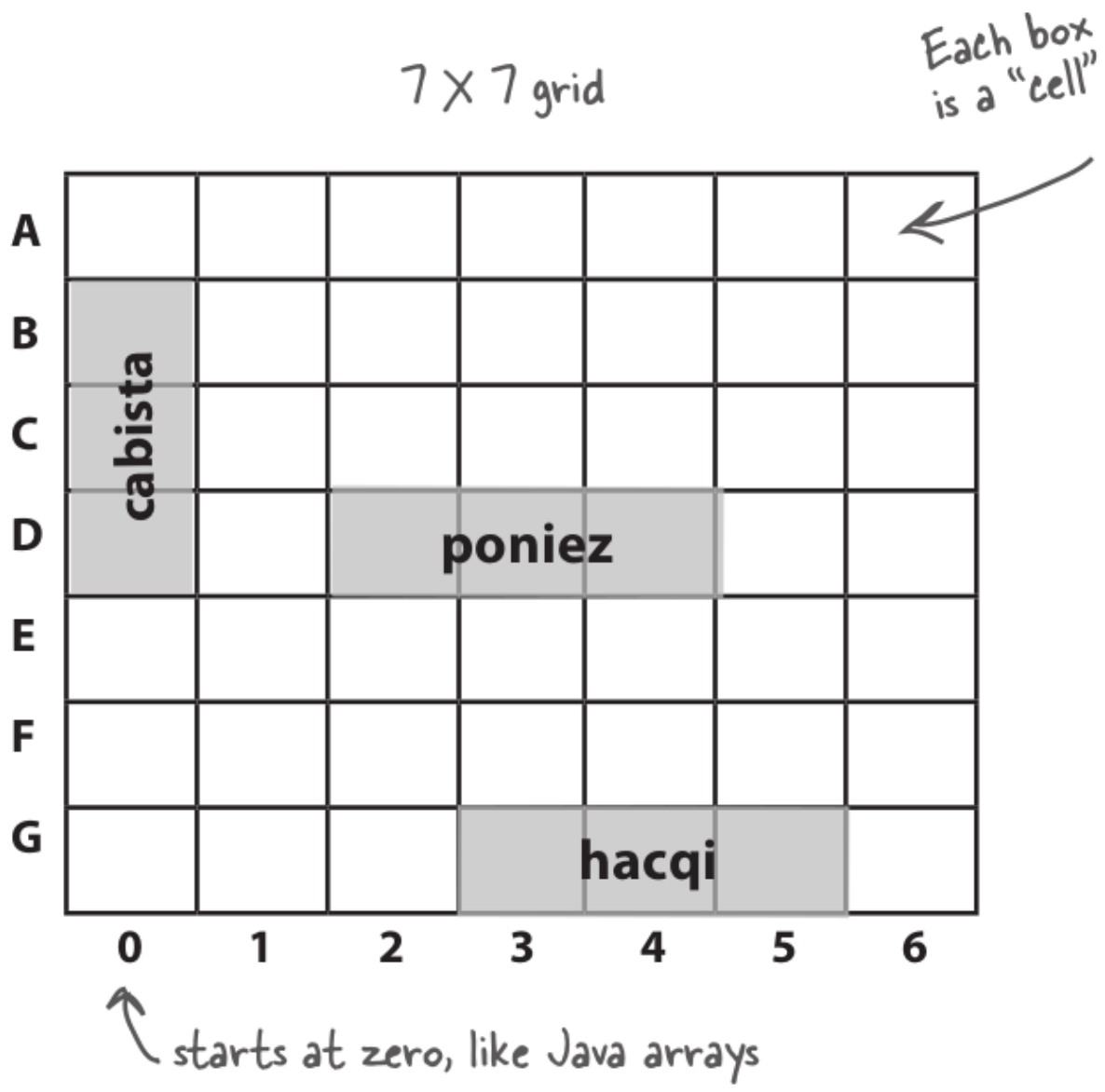
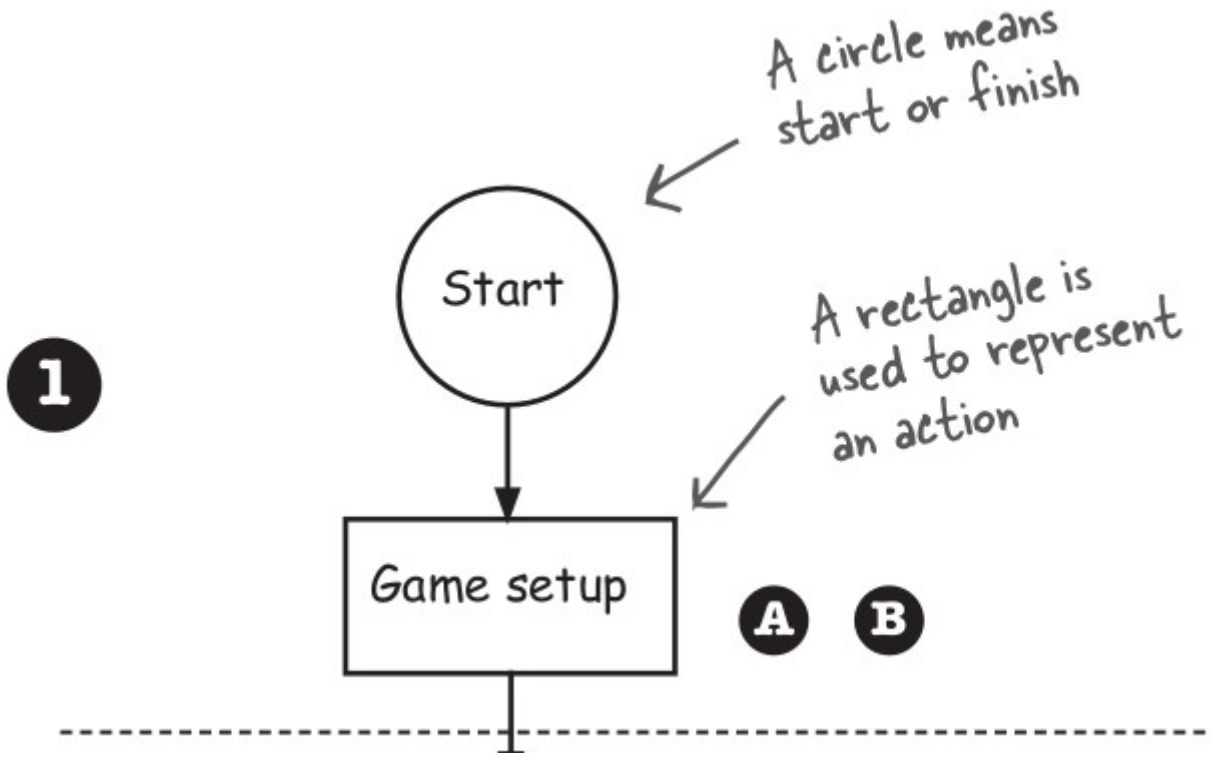
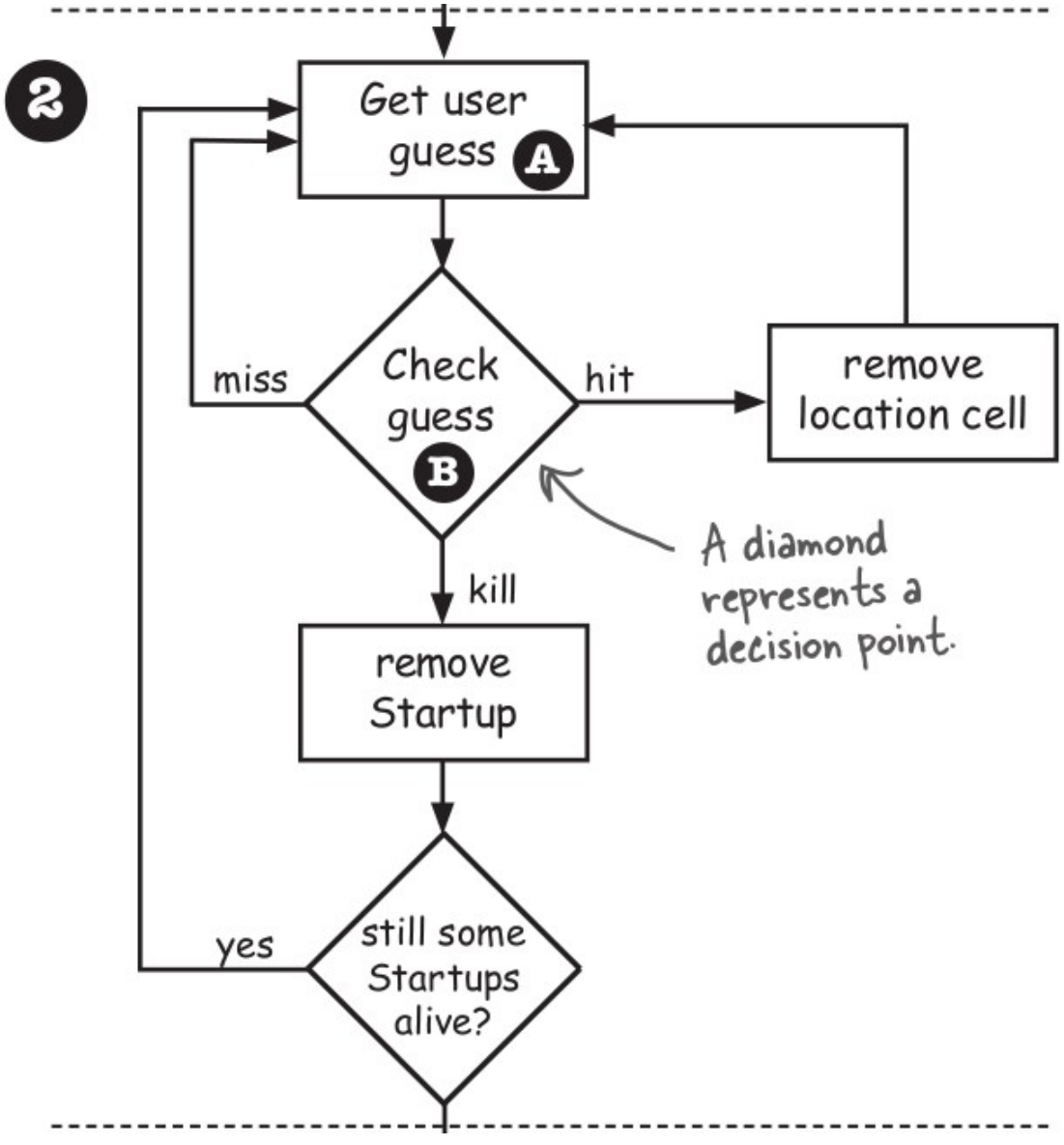
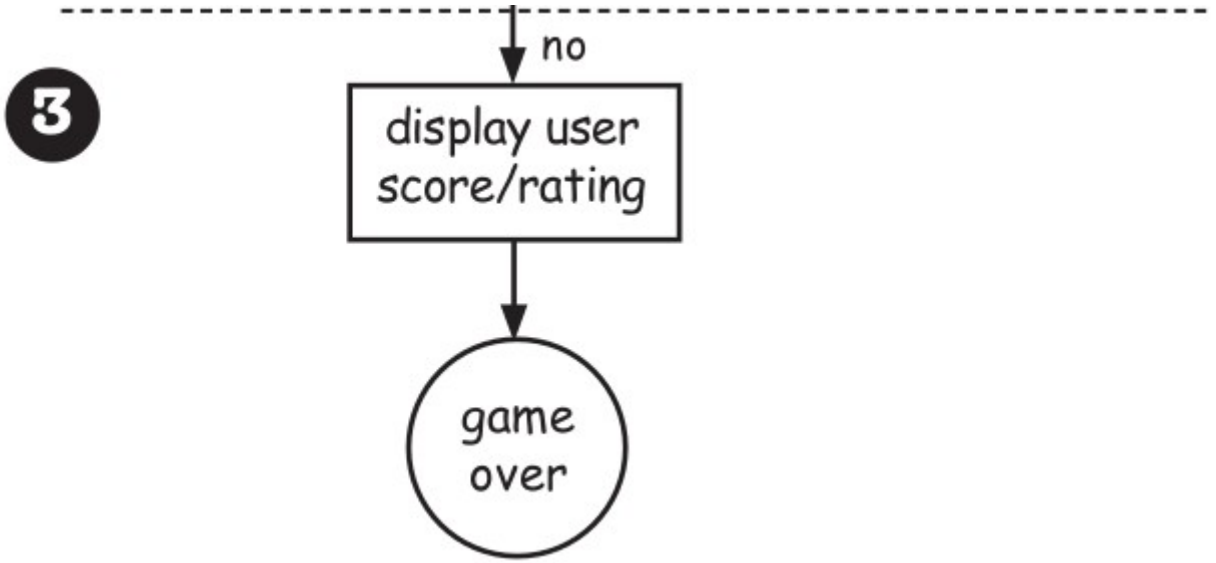

I can lift
heavy objects.

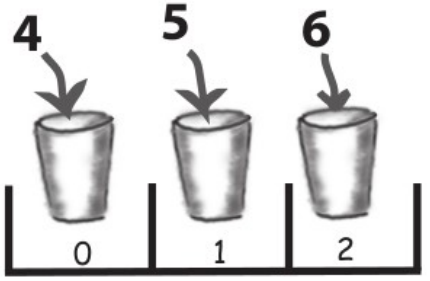

7 X 7 grid
Each box is a "cell"
A
B
C
D
E
F
G
cabista
poniez
hacqi
0
1
2
3
4
5
6
starts at zero, like Java arrays


A circle means start or finish
Start
A rectangle is used to represent an action
1
Game setup
A
B


2
Get user guess
A
Check guess
B
miss
hit
remove location cell
A diamond represents a decision point.
kill
remove Startup
still some Startups alive?
yes

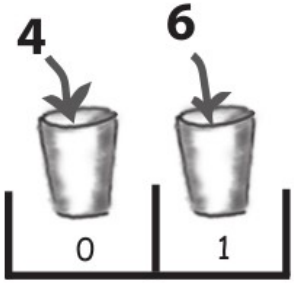

3
no
display user score/rating
game over


**locationCells array** BEFORE any cells have been hit




The array starts out with a size of 3, and we loop through all 3 cells (positions in the array) to look for a match between the user guess and the cell value (4,5, 6).

**locationCells array** AFTER cell '5', which was at index 1 in the array, has been hit




When cell '5' is hit, we make a new, smaller array with only the remaining cell locations, and assign it to the original locationCells reference.


If only I could find an array that could **shrink** when you **remove** something. And one that you didn't have to loop through to check each element, but instead you could just **ask it if it contains** what you're looking for. And it would let you **get** things out of it, without having to know exactly which slot the things are in. That would be dreamy. But I know it's just a fantasy...

# Using the Java Library

## ArrayList

**add(E e)**
Appends the specified element to the end of this list.

**remove(int index)**
Removes the element at the specified position.

**remove(Object o)**
Removes the first occurrence of the specified element.

**contains(Object o)**
Returns true if this list contains the specified element.

**isEmpty()**
Returns "true" if the list contains no elements.

**indexOf(Object o)**
Returns either the first index of the element, or -1.

**size()**
Returns the number of elements in this list.

**get(int index)**
Returns the element at the specified position.

# Some things you can do with ArrayList

Don't worry about this new <Egg> angle-bracket syntax right now; it just means "make this a list of Egg objects."

1. Make one

```
ArrayList<Egg> myList = new ArrayList<Egg>();
```

A new ArrayList object is created on the heap. It's little because it's empty.

2. Put something in it

```
Egg egg1 = new Egg();

myList.add(egg1);
```


Now the ArrayList grows a "box" to hold the Egg object.

3. Put another thing in it

```
Egg egg2 = new Egg();

myList.add(egg2);
```


The ArrayList grows again to hold the second Egg object.

4. Find out how many things are in it

```
int theSize = myList.size();
```

The ArrayList is holding 2 objects so the size() method returns 2.

5. Find out if it contains something

```
boolean isIn = myList.contains(egg1);
```

The ArrayList DOES contain the Egg object referenced by 'egg1', so contains() returns **true**.

6. Find out where something is (i.e., its index)

```
int idx = myList.indexOf(egg2);
```

ArrayList is zero-based (means first index is 0) and since the object referenced by 'egg2' was the second thing in the list, indexOf() returns **1**.

7. Find out if it's empty

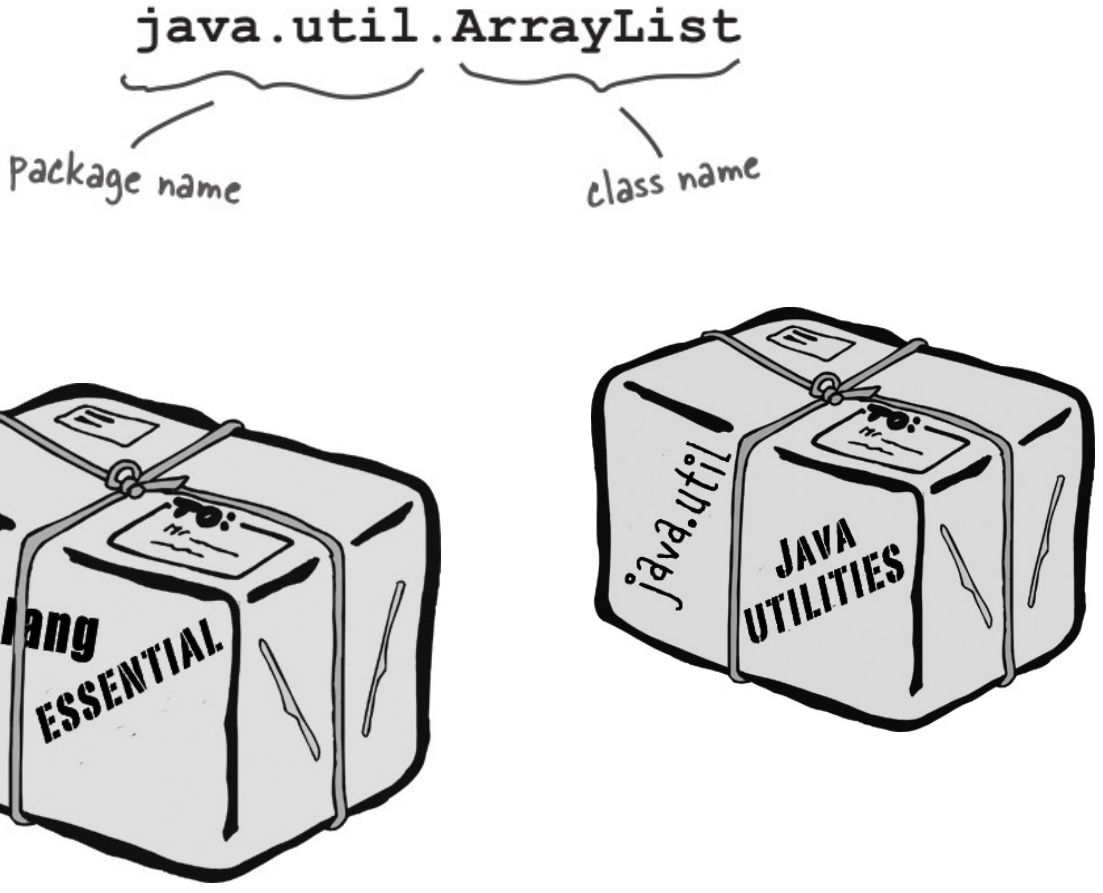
```
boolean empty = myList.isEmpty();
```

it's definitely NOT empty, so isEmpty() returns **false**.

8. Remove something from it

```
myList.remove(egg1);
```


Hey look — it shrank!


java.util.ArrayList
Package name
class name
java.lang
ESSENTIAL
java.util
JAVA
UTILITIES

## IMPORT

**A** Put an import statement at the top of your source code file:

```
import java.util.ArrayList;
public class MyClass {... }
```

**OR**

## TYPE

**B** Type the full name everywhere in your code. Each time you use it.
*Everywhere* you use it.

When you declare and/or instantiate it:

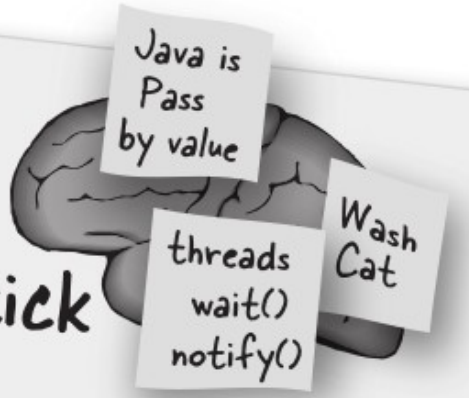
```
java.util.ArrayList<Dog> list = new java.util.ArrayList<Dog>();
```

When you use it as an argument type:

```
public void go(java.util.ArrayList<Dog> list) { }
```

When you use it as a return type:

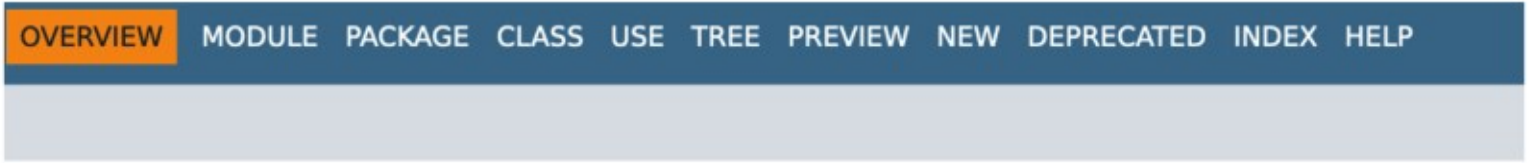
```
public java.util.ArrayList<Dog> foo() {...}
```




# Make it Stick

*Roses are red,*
*apples are ripe,*
*if you don't import*
*you'll just have to type*

**You must tell Java the full name of every class you use**, unless that class is in the java.lang package. An `import` statement for the class or package at the top of your source code is the easy way. Otherwise, you have to type the full name of the class, everywhere you use it!




# Java® Platform, Standard Edition & Java Development Kit Version 17 API Specification

This document is divided into two sections:

Java SE

The Java Platform, Standard Edition (Java SE) APIs define the core Java platform

JDK

The Java Development Kit (JDK) APIs are specific to the JDK and will not necessar

**All Modules** | **Java SE** | **JDK** | **Other Modules**

| Module | Description |
|---|---|
| java.base | Defines the foundational APIs of the Java SE Platform. |
| java.compiler | Defines the Language Model, Annotation Processing, a |
| java.datatransfer | Defines the API for transferring data between and with |

***https://docs.oracle.com/en/java/javase/17/docs/api/index.html***